



Doom  [rspress](#)  Alauda  [rspress](#)  Alauda  [rspress](#) Alauda



□ □ □ □

□□ Markdown □□□□ MDX

□ □ □ □



- □□□□□□□□□□□□□□□□
- □□□□□□□
- □□□□□

□□ **Markdown** □□□□ **MDX**



0000000000 MDX 000 ↗ 0

0000

000000000000 Doom 00

00

00

0000

000000

00

Markdown

Callouts

Mermaid

📄📄📄

MDX

📄 MDX 📄📄📄📄📄📄📄📄📄📄

rspress 📄

doom 📄

📄📄📄📄📄

📄📄

📄📄📄📄📄📄📄📄📄

i18n.json

.ts/.tsx

.mdx

API 📄

📄 API

CRD

📄📄📄

📄 openapi 📄

📄📄📄📄📄

props

📄

0000

000000

00

00000000000000000000 ACP 00

00000

00000

000000

00000000

📄📄

📄📄

📄📄📄

📄📄📄📄

📄📄📄📄📄

📄📄📄📄📄

📄📄📄

📄📄📄📄📄📄

📄📄📄

📄📄 PDF

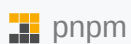
📄📄📄

📄📄📄📄

📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄

```
mkdir my-docs && cd my-docs
```

📄 `npm init -y` 📄📄📄📄📄📄📄📄📄📄📄📄 `npm`📄`yarn` 📄 `pnpm` 📄📄 `doom:`



```
npm install -D @alauda/doom typescript
```

📄📄📄📄📄📄📄📄:

```
# 创建 docs 目录并初始化
mkdir docs/en && echo '# Hello World' > docs/en/index.md
mkdir docs/zh && echo '# 文档' > docs/zh/index.md
```

在 `package.json` 中添加脚本:

```
{
  "scripts": {
    "dev": "doom dev",
    "build": "doom build",
    "new": "doom new",
    "serve": "doom serve",
    "translate": "doom translate",
    "export": "doom export"
  }
}
```

在 `doom.config.yml` 中添加:

```
title: My Docs
```

在 `tsconfig.json` 中添加:

```
{
  "compilerOptions": {
    "jsx": "react-jsx",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "resolveJsonModule": true,
    "skipLibCheck": true,
    "strict": true,
    "target": "ESNext",
  },
  "mdx": {
    "checkMdx": true,
  },
}
```

global.d.ts

```
/// <reference types="@alauda/doom/runtime" />
```

.mdx doom


```
doom -h
```

```
# output
```

```
Usage: doom [options] [command]
```

```
Doctor Doom making docs.
```

```
Options:
```

```

  -V, --version                output the version number
  -c, --config <config>       Specify the path to the config file
  -v <version>                 Specify the version of the
documentation, can also be `unversioned` or `unversioned-x.y`
  -b, --base <base>            Override the base of the documentation
  -p, --prefix <prefix>        Specify the prefix of the documentation
base
  -f, --force [boolean]        Force to
                                1. fetch latest reference remotes or
scaffolding templates, otherwise use local cache
                                2. translate ignore hash equality check
and original text (default: false)
  -i, --ignore [boolean]       Ignore internal routes (default: false)
  -d, --download [boolean]     Display download pdf link on nav bar
(default: false)
  -e, --export [boolean]       Run or build in exporting PDF mode,
`apis/**` and `*/apis/**` routes will be ignored automatically (default:
false)
  -I, --include <language...> Include **only** the specific
language(s), `en ru` for example
  -E, --exclude <language...> Include all languages except the
specific language(s), `ru` for example
  -o, --out-dir <path>         Override the `outDir` defined in the
config file or the default `dist/{base}/{version}`, the resulting path
will be `dist/{outDir}/{version}`
  -r, --redirect <enum>        Whether to redirect to the locale
closest to `navigator.language` when the user visits the site, could be
`auto`, `never` or `only-default-lang` (default: "only-default-lang")
  -R, --edit-repo [boolean|url] Whether to enable or override the
`editRepoBaseUrl` config feature, `https://github.com/` prefix could be
omitted (default: false)
  -a, --algolia [boolean|alauda] Whether to enable or use the alauda
(docs.alauda.io) preset for Algolia search (default: false)
  -S, --site-url                Whether to enable the siteUrl for
sitemap generation (default: false)

```

<code>-n, --no-open</code>	Do not open the browser after starting the server
<code>-h, --help</code>	display help for command
 Commands:	
<code>dev [options] [root]</code>	Start the development server
<code>build [root]</code>	Build the documentation
<code>preview serve [options] [root]</code>	Preview the built documentation
<code>new [template]</code>	Generate scaffolding from templates
<code>translate [options] [root]</code>	Translate the documentation
<code>export [options] [root]</code>	Export the documentation as PDF,
<code>`apis/**` and `*/apis/**` routes</code>	will be ignored automatically
<code>lint [root]</code>	Lint the documentation
<code>help [command]</code>	display help for command



□□ yarn dev □□□□□□□□□□□□□□□□□□□□

```
doom dev -h
```

output

```
Usage: doom dev [options] [root]
```

Start the development server

Arguments:

root	Root directory of the documentation
------	-------------------------------------

Options:

-H, --host [host]	Dev server host name
-P, --port [port]	Dev server port number
-l, --lazy [boolean]	Whether to enable <code>`lazyCompilation`</code> which could improve the compilation performance (default: <code>true</code>)
--no-lazy	Do not enable <code>`lazyCompilation`</code>
-h, --help	display help for command

```
# output
```

```
Usage: doom dev [options] [root]
```

Start the development server

Arguments:

```
root      Root directory of the documentation
```

Options:

```
-H, --host [host]      Dev server host name
-P, --port [port]      Dev server port number
-l, --lazy [boolean]    Whether to enable `lazyCompilation` which
                        would improve the compilation performance (default: true)
--no-lazy                Do not enable `lazyCompilation`
-h, --help              display help for command
```

--	--	--	--	--	--

`yarn build` 构建项目并生成 `dist` 目录下的文件

使用

`yarn serve` 启动本地服务器 `-b`, `-p` 指定本地服务器端口 `-b`, `-p` 指定本地服务器端口

使用

`yarn new` 创建新项目

使用

```
doom translate -h
```

```
# output
```

```
Usage: doom translate [options] [root]
```

```
Translate the documentation
```

```
Arguments:
```

```
root
```

```
Root directory of the documentation
```

```
Options:
```

```
-s, --source <language> Document source language, one of en, zh, ru
(default: "en")
```

```
-t, --target <language> Document target language, one of en, zh, ru
(default: "zh")
```

```
-g, --glob <path...>
```

```
Glob patterns of source dirs/files to translate
```

```
-C, --copy [boolean]
```

```
Whether to copy relative assets to the target
```

```
directory instead of following links (default: false)
```

```
-h, --help
```

```
display help for command
```

- `-g, --glob` 指定要翻译的源文件的路径 `glob` 指定要翻译的源文件的路径

- `yarn translate -g abc xyz` 指定 `<root>/<source>/abc`, `<root>/<source>/xyz` 指定 `<root>/<target>/abc`, `<root>/<target>/xyz` 指定

2. `yarn translate -g '*'` `<root>/<source>` `<root>/<target>`

- `-C, --copy` `<root>/<source>` `<root>/<target>` `false` `<root>/<target>`

- `<root>/<target>`

1. `<root>/<source>/abc.jpg` `<root>/public/<source>/abc.jpg` `<root>/public/<target>/abc.jpg` `<root>/<target>/abc.jpg`
2. `<root>/<source>/abc.mdx` `<root>/assets/xyz.jpg` `<root>/<source>/assets/xyz.jpg` `<root>/<target>/assets/xyz.jpg`
3. `<root>/<source>/abc.mdx` `<root>/assets/<source>/xyz.jpg` `<root>/<source>/assets/<source>/xyz.jpg` `<root>/<target>/assets/<target>/xyz.jpg` `<root>/assets/<target>/xyz.jpg`

- `<root>/<target>`

1. `<root>/<source>/abc.jpg` `<root>/public/<target>/abc.jpg` `<root>/<target>/abc.jpg`
2. `<root>/<source>/abc.mdx` `<root>/assets/<source>/xyz.jpg` `<root>/<target>/assets/<target>/xyz.jpg` `<root>/assets/<target>/xyz.jpg` `<root>/assets/<target>/xyz.jpg`

WARNING

`<root>/<source>` `-g '*'` `<root>/<source>` `source` `<root>/<target>` `target` `<root>/<target>` `internalRoutes` `<root>/<target>` `target` `<root>/<target>`

TIP

`<root>/<source>` `AZURE_OPENAI_API_KEY` `<root>/<source>` `Leader` `<root>/<source>`

`<root>/<source>` `<root>/<source>`

```
il8n:
  title:
    en: DevOps Connectors
  additionalPrompts: '📄📄 Connectors 📄📄📄📄📄📄'
  disableAutoTranslation: false
title: DevOps 📄
```

📄📄📄📄📄📄📄📄

📄 PDF

WARNING

📄📄📄📄📄📄📄📄 `yarn build` 📄📄

```
doom export -h
```

```
# output
```

```
Usage: doom export [options] [root]
```

Export the documentation as PDF, ``apis/**`` and ``*/apis/**`` routes will be ignored automatically

Arguments:

root Root directory of the documentation

Options:

-H, --host [host] Serve host name

-P, --port [port] Serve port number (default: "4173")

-h, --help display help for command

📄 `yarn export` 📄📄 PDF 📄📄📄📄📄📄 `-b` , `-p` 📄📄📄📄📄📄📄📄 `-b` , `-p`
📄

📄📄📄📄 `playwright` ➔📄📄📄📄 `build-harbor.alauda.cn/frontend/playwright-`
`runner:doom` 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄

```
.env.yarn
```

```
PLAYWRIGHT_DOWNLOAD_HOST="https://cdn.npmmirror.com/binaries/playwright"
```

pdf doom pdf

```
doom lint -h
```

```
# output
```

```
Usage: doom lint [options] [root]
```

```
Lint the documentation
```

```
Arguments:
```

```
  root                Root directory of the documentation
```

```
Options:
```

```
  -g, --glob <path...>  Glob patterns of source dirs/files to lint
                        (default: "**/*.js,jsx,ts,tsx,md,mdx")
  --no-cspell             Disable cspell linting
  -h, --help             display help for command
```

```
doom lint ESLint cspell
```

ESLint / CSPELL:

- eslint.config.mjs:

```
import doom from '@alauda/doom/eslint'

export default doom(new URL('docs', import.meta.url))
```

- cspell.config.mjs:

```
export { default } from '@alauda/doom/cspell'
```

📁📁📁📁📁📁📁📁📁 CWD 📁 .cspell 📁📁📁📁📁 CSpell 📁📁📁📁📁📁📁📁
📁.cspell/k8s.txt 📁

k8s
kubernetes

📁📁📁📁📁📁📁📁📁📁



📄

📄 `doom` 📄📄

📄📄

📄📄

API 📄📄

📄📄📄📄📄

📄📄📄

📄📄📄

📄📄📄

📄📄📄📄📄

📄📄📄📄📄📄

📄📄📄📄📄

`sites.yaml` 📄

📄📄

📄📄📄📄📄📄

📄📄📄

📄📄📄

Algolia 📄📄

Sitemap 📄

📄

📄“📄📄📄📄”📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄

📄📄📄

📄📄

📄

📄

Markdown

Callouts

Mermaid

📄📄📄

MDX

📄 MDX 📄📄📄📄📄📄📄📄📄📄

rspress 📄

doom 📄

📄📄📄📄📄

📄📄

📄📄📄📄📄📄📄📄

i18n.json

.ts/.tsx

.mdx

API 00

00 API

CRD

0000

00 openapi 00

000000

props

00

0000

000000

00

000000000000000000000000 ACP 00

00000

00000

000000

00000000

📁📁

📁📁

📁📁📁

📁📁📁

API 📁📁📁

📁📁📁📁📁📁

📁📁📁📁📁

frontmatterMode

📁📁📁📁📁

📁📁📁📁

📁📁📁📁📁📁

📁📁📁📁📁📁📁

📁📁📁📁📁📁

sites.yaml 📁📁

📁📁📁

📁📁📁📁📁📁📁

📁📁📁📁📁

📁📁📁📁📁

Algolia 📁📁📁

Sitemap 📁📁

📁📁📁📁

```

##### yaml ##### doom.config.yaml []

doom.config.yaml ##### rspress ##### js/ts #####

##### .js/.ts/.mjs/.mts/.cjs/.cts #####

[] js/ts ##### @alauda/doom/config ##### defineConfig []

#####

```

```
import { defineConfig } from '@alauda/doom/config'

export default defineConfig({})
```

--	--	--	--

- `lang` 指定する言語 (デフォルトは `en`)
`null` ない `undefined`
- `title` タイトル
- `logo` ロゴ (デフォルトは `public`)
`doom` ない `alauda logo`
- `logoText` ロゴのテキスト
- `icon` アイコン (デフォルトは `logo`)
- `base` ベースパス (デフォルトは `product-docs`)
/
- `outDir` 出力ディレクトリ (デフォルトは `dist/{base}/{version}`)
`dist/{outDir}/{version}` ない `version` ない

API ☐ ☐ ☐ ☐

api:

```
# CRD 配置文件 doom.config.* 文件 glob 模式 json/yaml 文件
```

crds:

```
- docs/shared/crds/*.yaml
```

```
# OpenAPI 配置文件 doom.config.* 文件 glob 模式 json/yaml 文件
```

openapis:

```
- docs/shared/openapis/*.json
```

```
# 为 openapi 文件生成 OpenAPI 规范文档
```

```
# 为
```

<https://doom.alauda.cn/apis/references/CodeQuality.html#v1alpha1.CodeQualityS>

references:

```
  v1alpha1.CodeQualityBranch:
```

</apis/references/CodeQualityBranch#v1alpha1.CodeQualityBranch>

```
# 为 API 文件生成 OpenAPI 规范文档 gateway 文件
```

```
pathPrefix: /apis
```

API 文档

API 文档

```
# 配置文件 doom.config.* 文件 glob 模式 json/yaml 文件
```

permission:**functionresources:**

```
# `kubectl get functionresources`
```

```
- docs/shared/functionresources/*.yaml
```

roletemplates:

```
# `kubectl get roletemplates -l
```

```
auth.cpaas.io/roletemplate.official=true`
```

```
- docs/shared/roletemplates/*.yaml
```

API 文档

API 文档

reference:

```

- repo: alauda-public/product-doc-guide #
branch: # [string]
publicBase: # [string] /images/xx.png
docs/public
sources:
- name: anchor #
path: docs/index.mdx# #
doom.config.*
ignoreHeading: # [boolean] true
processors: #
- type: ejsTemplate
data: # ejs `<%= data.xx %>`
frontmatterMode: merge # frontmatter ignore
ignore/merge/replace/remove

```

frontmatterMode

- **ignore** frontmatter frontmatter
- **merge** frontmatter key
- **replace** frontmatter frontmatter
- **remove** frontmatter

releaseNotes:**queryTemplates:**

```

fixed: # ejs jql
unfixed:

```

release-notes.md

```
<!-- release-notes-for-bugs?template=fixed&project=DevOps -->
```

release-notes.mdx

```
{/* release-notes-for-bugs?template=fixed&project=DevOps */}
```

🐱🐱 `template=fixed&project=DevOps` 🐱🐱 `fixed` 🐱 `queryTemplates` 🐱🐱🐱🐱🐱🐱
🐱🐱 `query` 🐱🐱 `project=DevOps` 🐱🐱🐱 `ejs` ↗ 🐱🐱🐱🐱🐱 `fixed` 🐱🐱🐱🐱🐱 `jira` `jql` ↗
🐱🐱 `https://jira.alauda.cn/rest/api/2/search?jql=<jql>` 🐱🐱🐱 `API` 🐱🐱🐱🐱🐱🐱
`JIRA_USERNAME` 🐱 `JIRA_PASSWORD` 🐱🐱🐱🐱🐱🐱🐱

🐱🐱🐱🐱🐱

`sidebar:`
 `collapsed: false` # 🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱 `false`

🐱🐱🐱🐱🐱🐱🐱

`internalRoutes:` # 🐱🐱🐱🐱 `glob` 🐱🐱🐱🐱 `docs` 🐱🐱🐱 `cli` 🐱🐱 ``-i, --ignore`` 🐱🐱🐱🐱
🐱🐱🐱/🐱🐱🐱🐱
`- '*/internal/**'`

🐱🐱🐱🐱🐱🐱🐱🐱

```
onlyIncludeRoutes: # 只包含 glob 匹配的 docs 页面 cli 中 -i, --ignore 选项
                    匹配的路径/文件夹名称 `internalRoutes` 中指定的路径
    - '*/internal/**'
internalRoutes:
    - '*/internal/overview.mdx'
```



```
shiki:
  theme: # optional, https://shiki.style/themes
  langs: # optional, https://shiki.style/languages
  transformers: # optional, only available in js/ts config,
https://shiki.style/guide/transformers
```

WARNING

plaintext

`sites.yaml` ☐ ☐

[illegible]


```
- name: connectors # 连接器
base: /devops-connectors # 连接器
version: v1.1 # 连接器 ExternalSite/ExternalSiteLink 连接器

displayName: # 连接器 name
  en: DevOps Connectors
  zh: DevOps 连接器

# 连接器
# 连接器
repo: https://github.com/AlaudaDevops/connectors-operator # 连接器
gitlab slug: `alauda/product-docs`
image: devops/connectors-docs # 连接器
```

🔒🔒🔒🔒

translate:

```
# 将源语言 `sourceLang` 翻译成目标语言 `targetLang`，`userPrompt`，
`additionalPrompts`，`terms`，`titleTranslationPrompt`
# 将 `sourceLang` 翻译成 `targetLang` 的 `terms` 和 `title` 的翻译
# `userPrompt` 的翻译
# `additionalPrompts` 的翻译 `frontmatter.il18n` 的 `additionalPrompts`
的翻译
# `terms` 和 `titleTranslationPrompt` 的翻译 AI
的翻译
# 的翻译
```

systemPrompt: |

You are a professional technical documentation engineer, skilled in writing high-quality technical documentation in <%= targetLang %>. Please accurately translate the following text from <%= sourceLang %> to <%= targetLang %>, maintaining the style consistent with technical documentation in <%= sourceLang %>.

Baseline Requirements

- Sentences should be fluent and conform to the expression habits of the <%= targetLang %> language.
- Input format is MDX; output format must also retain the original MDX format. Do not translate the names of jsx components such as <Overview />, and do not wrap output in unnecessary code blocks.
- ****CRITICAL****: Do not translate or modify ANY link content in the document. This includes:
 - **URLs in markdown links**: [text](URL) - keep URL exactly as is
 - **Reference-style links**: [text][ref] and [ref]: URL - keep both ref and URL unchanged
 - **Inline URLs**: https://example.com - keep completely unchanged
 - **Image links**: ![alt](src) - keep src unchanged, but alt text can be translated
 - **Anchor links**: [text](#anchor) - keep #anchor unchanged
 - Any href attributes in HTML tags - keep unchanged
- Do not translate professional technical terms and proper nouns, including but not limited to: Kubernetes, Docker, CLI, API, REST, GraphQL, JSON, YAML, Git, GitHub, GitLab, AWS, Azure, GCP, Linux, Windows, macOS, Node.js, React, Vue, Angular, TypeScript, JavaScript, Python, Java, Go, Rust, etc. Keep these terms in their original form.
- The title field and description field in frontmatter should be translated, other frontmatter fields should retain and do not translate.
- Content within MDX components needs to be translated, whereas MDX component names and parameter keys do not.
- Do not modify or translate any placeholders in the format of

__ANCHOR_N__ (where N is a number). These placeholders must be kept exactly as they appear in the source text.

- Keep original escape characters like backslash, angle brackets, etc. unchanged during translation.
- Do not add any escape characters to special characters like [], (), {}, etc. unless they were explicitly present in the source text. For example:
 - If source has "Architecture [Optional]", keep it as "Architecture [Optional]" (not "Architecture \[Optional]")
 - If source has "Function (param)", keep it as "Function (param)" (not "Function \((param)")
 - Only add escape characters if they were present in the original text
- Preserve and do not translate the following comments, nor modify their content:
 - { /* release-notes-for-bugs */ }
 - <!-- release-notes-for-bugs -->
- Remove and do not retain the following comments:
 - { /* reference-start */ }
 - { /* reference-end */ }
 - <!-- reference-start -->
 - <!-- reference-end -->
- Ensure the original Markdown format remains intact during translation, such as frontmatter, code blocks, lists, tables, etc.
- Do not translate the content of the code block.

```
<% if (titleTranslationPrompt) { %>
<%- titleTranslationPrompt %>
<% } %>
<% if (terms) { %>
<%- terms %>
<% } %>
```

```
<% if (userPrompt || additionalPrompts) { %>
```

```
## Additional Requirements
```

These are additional requirements for the translation. They should be met along with the baseline requirements, and in case of any conflict, the baseline requirements should take precedence.

The text for translation is provided below, within triple quotes:

```
"""
```

```
<% if (userPrompt) { %>
<%- userPrompt %>
<% } %>
```

```
<% if (additionalPrompts) { %>
<%- additionalPrompts %>
```

```
<% } %>
""
<% } %>
```

```
editRepoBaseUrl: alauda/doom/tree/main/docs # https://github.com/
`-R, --edit-repo`
```

```
export:
  - name: Concepts # pdf
    scope: '*/concepts' # glob docs
```

```
lint:
  cspellOptions: # cspell
https://github.com/streetsidesoftware/cspell/tree/main/packages/cspell-
eslint-plugin#options
```

Algolia

```
algolia: # Algolia 配置参数 -a, --algolia` 配置参数
appId: # Algolia ID
apiKey: # Algolia API Key
indexName: # Algolia 索引名
```

public/robots.txt Algolia

INFO

[rspress](#)
[Algolia](#)
[🔗](#)
[@alauda/doom/theme](#)

theme/index.ts

```
export * from '@alauda/doom/theme'
```

Sitemap ☐ ☐

```
siteUrl: https://docs.alauda.cn # 指定 URL 生成 sitemap 文件 -S, --site-  
url` 指定生成 sitemap 文件的 URL
```

□ □ □ □

□ □ □

11

11

--	--	--	--

```

index
index.md index.mdx

```

```
├─ index.md
├─ start.mdx
└─ usage
    ├── index.mdx
    └─ convention.md
```

□ □ □ □ □

1. `public` □□□□□□□□□□□□□□□□
2. `public/_remotes` □□□□□□□□□□□□□□□□□□□□□□□□□□□□
`*/public/_remotes` □□ `.gitignore` □□□□□□□□
3. `shared` □□□□□□□□□□□□□□□□□□□□□□□□□□□□

```

---
title: []
description: []
author: []
category: []
---
```

```

- - -
weight: 1
- - -

```

[illegible]

index.mdx Overview

```
#  
  
<Overview />
```


1. `;;, %, #, //, /** */, --` `<!-- -->`
2. `[\\!code callout]`
3. `:::callouts` `<div class="doom-callouts">`
`<Callouts>`

```
```sh
```

```
Memory overhead per virtual machine ≈ (1.002 × requested memory) \
+ 218 MiB \ # [!code callout]
+ 8 MiB × (number of vCPUs) \ # [!code callout]
+ 16 MiB × (number of graphics devices) \ # [!code callout]
+ (additional memory overhead) # [!code callout]
```

```
```
```

```
:::callouts
```

1. Required for the processes that run in the `virt-launcher` pod.
2. Number of virtual CPUs requested by the virtual machine.
3. Number of virtual graphics cards requested by the virtual machine.
4. Additional memory overhead:
 - If your environment includes a Single Root I/O Virtualization (SR-IOV) network device or a Graphics Processing Unit (GPU), allocate 1 GiB additional memory overhead for each device.
 - If Secure Encrypted Virtualization (SEV) is enabled, add 256 MiB.
 - If Trusted Platform Module (TPM) is enabled, add 53 MiB.

```
:::
```

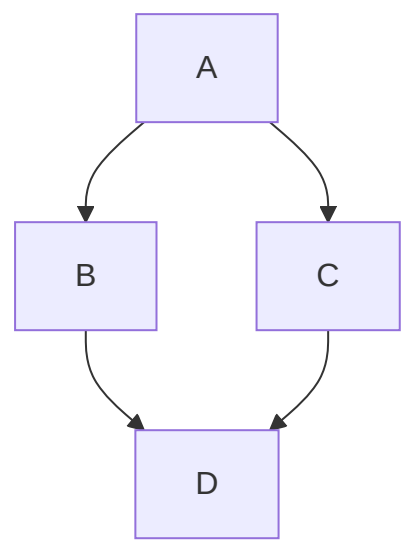
```
Memory overhead per virtual machine ≈ (1.002 × requested memory) \
+ 218 MiB \ ①
+ 8 MiB × (number of vCPUs) \ ②
+ 16 MiB × (number of graphics devices) \ ③
+ (additional memory overhead) ④
```

- ① Required for the processes that run in the `virt-launcher` pod.
- ② Number of virtual CPUs requested by the virtual machine.
- ③ Number of virtual graphics cards requested by the virtual machine.
- ④ Additional memory overhead:
 - If your environment includes a Single Root I/O Virtualization (SR-IOV) network device or a Graphics Processing Unit (GPU), allocate 1 GiB additional memory overhead for each device.
 - If Secure Encrypted Virtualization (SEV) is enabled, add 256 MiB.
 - If Trusted Platform Module (TPM) is enabled, add 53 MiB.

Shiki Transformers ↗

Mermaid ↗

```
mermaid
graph TD;
  A-->B;
  A-->C;
  B-->D;
  C-->D;
```



Markdown Preview Mermaid ↗ VSCode

WARNING

.md [issue ↗](#)

```
{width="100" height="100"}
```



MDX

[MDX](#) [Markdown](#) [Markdown](#) [JSX](#) [rspress MDX](#)

rspress

doom

- Overview
- Directive
- ExternalSite
- ExternalSiteLink
- AcpApisOverview ExternalApisOverview

Term

- props
- TermsTable
- props
- JsonViewer

rspress

rspress [rspress](#) [.mdx](#)

- Badge
- Card

- [LinkCard](#)
- [PackageManagerTabs](#)
- [Steps](#)
- [Tab/Tabs](#)
- [Toc](#)

在 `rspress/theme` 中配置

preview.mdx

```
import { SourceCode } from '@rspress/core/theme'

<SourceCode href="/" />
```

doom 文档

`doom` 是一个用于生成文档的工具

Overview

文档生成工具

Directive

在文档中使用 `Directive` 来定义文档结构

```
- 在文档中定义 Directive 来定义文档结构

<Directive type="danger" title="危险">
  这里是一个危险的示例
  </Directive>
```

- 在 `( doc/en )` 中，`doc/zh` 是默认值，因为它是第一个值。

在

在 `doc/zh` 中，`doc/zh` 是默认值，因为它是第一个值。

在

ExternalSite

在

```
<ExternalSite name="connectors" />
```

Note

在 DevOps 中，`devops` 是默认值，因为它是第一个值。

[DevOps 文档](#) ↗

ExternalSiteLink

在

```
<ExternalSiteLink name="connectors" href="link.mdx#hash"
children="Content" />
```

[Content](#) ↗

TIP

在 mdx 中，`<ExternalSiteLink name="connectors" href="link" children="Content" />`

在

```
<ExternalSiteLink name="connectors" href="link">
  Content {/* `p` */}
</ExternalSiteLink>
```

p children

AcpApisOverview ExternalApisOverview

API

```
<AcpApisOverview />
{/* same as following */}
<ExternalApisOverview name="acp" />

<ExternalApisOverview name="connectors" />
```

Note

ACP APIs [ACP APIs](#)

Note

DevOps APIs [DevOps APIs](#)

Term

```
<Term name="company" textCase="capitalize" />
<Term name="product" textCase="lower" />
<Term name="productShort" textCase="upper" />
```

ACP

props

- `name` : □□□□□□□□□□□□□□□□
- `textCase` : □□□□□□□□□□□□ `lower` , `upper` , `capitalize`

TermsTable



```
<TermsTable />
```

公司	名称	类型	名称	类型	名称
company	名称	-	Alauda	-	名称
product	名称 名称	-	Alauda Container Platform	-	名称
productShort	ACP	-	ACP	-	名称 名称

props

- terms : NormalizedTermItem[] □□□□□□□□□□□□□□□□□□□□□□□□

JsonViewer

```
<JsonViewer value={{ key: 'value' }} />
```

```
yaml      json
key:    value
```

□□□□□□□□

□□□□□□□□□□□□□□□□□□ `shared` □□□□□□□□□□□□□□□□□□□□

```
import CommonContent from './shared/CommonContent.mdx'

<CommonContent />
```

□□□□□□□□ [runtime](#) ↗ □□□ API□□□□□□ `.jsx/.tsx` □□□□□□□□ `.mdx` □□□□□□□□

```
// shared/CommonContent.tsx
export const CommonContent = () => {
  const { page } = usePageData()
  return <div>{page.title}</div>
}

// showcase/content.mdx
import { CommonContent } from './shared/CommonContent'
<CommonContent />
```

WARNING

□□□□□□ `.mdx` □□□□□□□□□□ `props` □□□□□□ [issue](#) ↗ □□□□□□□□ `props` □□□□□□□□
`.jsx/.tsx` □□□□□□□□

📄📄📄

`alauda` 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄 `en / zh` 📄📄📄📄📄📄📄📄📄📄📄📄
📄 `public` 📄📄📄 `en / zh` 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄

📄📄

- `i18n.json`
- `.ts/.tsx`
- `.mdx`

`i18n.json`

📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄 `docs` 📄📄📄 `i18n.json` 📄📄
📄📄📄📄📄 `useI18n` 📄📄📄📄📄📄📄📄

docs/i18n.json

```
{
  "title": {
    "zh": "标题",
    "en": "Title"
  },
  "description": {
    "zh": "描述",
    "en": "description"
  }
}
```

.ts/.tsx

```
import { useI18n } from '@rspress/core/runtime'

export const CommonContent = () => {
  const t = useI18n()
  return <h1>{t('title')}</h1>
}
```

.mdx

```
import { useI18n } from '@rspress/core/runtime'

# {useI18n()}('title')}

{useI18n()}('description')}
```

API 文档

API 文档 API 文档 CRD (Custom Resource Definition) 文档

```
├─ apis
│   ├── advanced-apis # API
│   ├── crds # CRDs
│   └── references # 文档
```

文档

API

props

CRD

props

文档

props

openapi 文档

API

advanced-apis/codeQualityTaskSummary.mdx

```
# CodeQualityTaskSummary

<OpenAPIPath path="/plugins/v1alpha1/template/codeQuality/task/{task-id}/summary" />
```

文档 CodeQualityTaskSummary

props

- path : OpenAPI schema paths
- pathPrefix : api.pathPrefix
- openapiPath : openapi

CRD

crds/ArtifactCleanupRun.mdx

```
# ArtifactCleanupRun

<K8sCrd name="artifactcleanupruns.artifacts.katanomi.dev" />
```

文档 ArtifactCleanupRun

props

- name : CRD metadata.name
- crdPath : openapi CRD

references/CodeQuality.mdx

CodeQuality

```
<OpenAPIRef schema="v1alpha1.CodeQuality" />
```

CodeQuality

props

- `schema` : OpenAPI schema `definitions` (v2) or `component/schemas` (v3) から取得
- `openapiPath` : `openapi` のパス

openapi

`OpenAPIPath` は `OpenAPIRef` が参照する `openapi` のパスを指定する。
`openapi` のパスは `openapiPath` で指定する。

```
<OpenAPIPath  
  path="/plugins/v1alpha1/template/codeQuality/task/{task-id}/summary"  
  openapiPath="shared/openapis/katanomi.json"  
>
```

0000000

```

<K8sPermissionTable functions={['devops-testplans', 'devops-testmodules']}
/>

```

00

props

00

props

- functions : string[] - 00000000 FunctionResource 000000

00

00	00	000 00	000 000	000 00	0000 000	00 00	000 00
0000 devops- testplans	00	✓	✓	✓	✓	✓	×
	00	✓	×	✓	✓	✓	×
	00	✓	×	✓	✓	✓	×
	00	✓	×	✓	✓	✓	×

00	00	000 00	000 000	000 00	0000 000	00 00	000 00
0000 devops- testmodules	00	✓	✓	✓	✓	✓	×
	00	✓	×	✓	✓	✓	×
	00	✓	×	✓	✓	✓	×
	00	✓	×	✓	✓	✓	×

☐ Markdown ☐☐☐☐

```
<!-- reference-end -->
```

```
{/* reference-end */}
```

```
□□□□□ <!-- reference-start#ref --> □□□□□
```

frontmatterMode



reference:

```

- repo: alauda-public/product-doc-guide # 文档仓库
  branch: # [string] 分支
  publicBase: # [string] 文档仓库根目录 /images/xx.png 文档仓库根目录
docs/public
sources:
  - name: anchor # 锚点
    path: docs/index.mdx# # 锚点
doom.config.*
  ignoreHeading: # [boolean] 是否忽略 heading true 忽略
  processors: # 处理器
    - type: ejsTemplate
      data: # ejs 模板 `<%= data.xx %>` 模板
      frontmatterMode: merge # 是否忽略 frontmatter 忽略 ignore
ignore/merge/replace/remove

```

frontmatterMode

- **ignore** 忽略 frontmatter
- **merge** 合并 frontmatter key
- **replace** 替换 frontmatter
- **remove** 移除 frontmatter

文档仓库

```
doom build # 
doom serve # 
```

doom build dist -v

```
# 4.0 release-4.0 4.0
doom build -v 4.0 # 4.0 dist/4.0 {base}/4.0
doom build -v master # master dist/master {base}/master
doom build -v {other} # dist/{other} {base}/{other}

# unversioned unversioned-x.y
doom build -v unversioned # dist/unversioned {base}
doom build -v unversioned-4.0 # 4.0 dist/unversioned {base}
```

□□□□□□

```
|— console-platform
|   |— 4.0
|   |— 4.1
|   |— index.html
|   |— overrides.yaml
|   |— versions.yaml
|— console-devops-docs
|   |— 4.0
|   |— 4.1
|   |— index.html
|   |— overrides.yaml
|   |— versions.yaml
|— console-tekton-docs
|   |— 1.0
|   |— 1.1
|   |— index.html
|   |— overrides.yaml
|   |— versions.yaml
```

index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Redirecting...</title>
    <meta http-equiv="refresh" content="0; url=/console-docs/4.1" />
  </head>
  <body>
    <p>Redirecting to <a href="/console-docs/4.1">/console-docs/4.1</a>
  </p>
  </body>
</html>
```

配置

overrides.yaml

```
# 配置
title:
  en: Doom - Alauda
  zh: Doom - 配置
logoText:
  en: Doom - Alauda
  zh: Doom - 配置
```

versions.yaml

```
- '4.1'
- '4.0'
```